

Computer Networks Course Project

Phase 1: Build and Observe the Network – Team 1

Vikrant (Mac 1) Asad (Mac 2) Mayank (Mac 3) Prashant (Mac 4)

1 Team Introduction and Setup Flow

We built a small private service platform on four MacBooks connected to the same Wi-Fi (network 10.7.0.0/19, gateway 10.7.0.1). A client types a private name, `app.team1.test`; our own DNS server resolves it; the client opens an HTTPS connection to our nginx edge; and nginx forwards the request to one of two backend servers.

Machine	Person	Role	Service / port	IPv4	Interface
Mac 1	Vikrant	Private DNS server	dnsmasq, UDP 53	10.7.13.235	en0
Mac 2	Asad	Edge proxy + load balancer	nginx, TCP 8443	10.7.17.39	en0
Mac 3	Mayank	Backend A + client	Python REST, 3001	10.7.5.187	en0
Mac 4	Prashant	Backend B + client	Python REST, 3002	10.7.15.184	en0

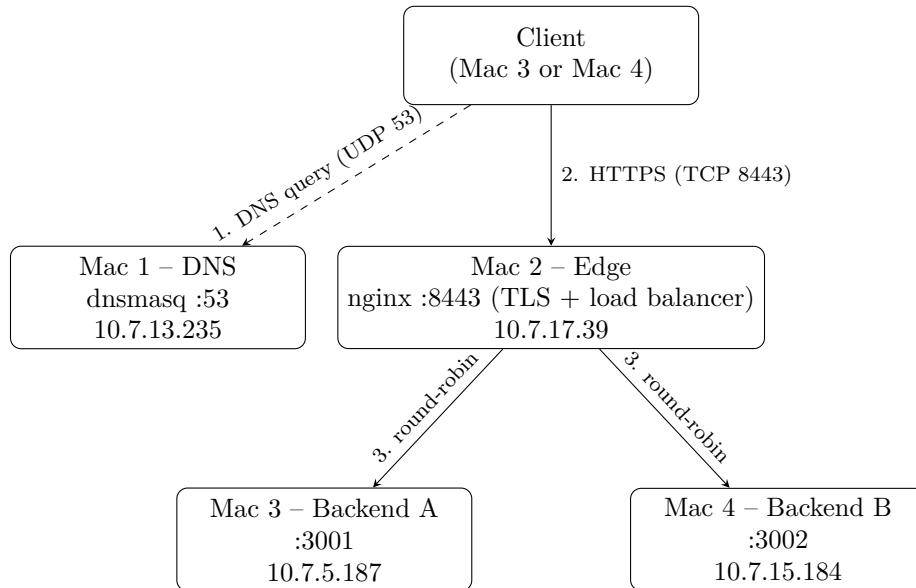


Figure 1: Network topology and request flow.

Setup flow (Task A and Task B)

1. All four Macs joined the same Wi-Fi. We recorded each Mac's interface, IPv4 address, mask (255.255.224.0 = /19), gateway and MAC address (full table in `docs/ARCHITECTURE.md`).
2. We checked reachability by pinging every pair of machines: 4 packets, 0% packet loss each.
3. Mac 1 runs dnsmasq with records for `app.team1.test` and `api.team1.test` pointing to the edge (Mac 2). Mac 3 and Mac 4 use Mac 1 as their DNS resolver.

```

mayankpillai@Mayanks-MacBook-Pro-2 CN Project % ./scripts/ping-matrix.sh
OK    Mac1-DNS      10.7.13.235
OK    Mac2-Edge     10.7.17.39
OK    Mac3-BackendA 10.7.5.187
OK    Mac4-BackendB 10.7.15.184
mayankpillai@Mayanks-MacBook-Pro-2 CN Project %

```

Figure 2: Task A: Mac 3 can reach all four machines (all OK).

```

[vikrantyadav@Vikrants-MacBook-Pro CN-Project % ping -c 4 10.7.17.39
PING 10.7.17.39 (10.7.17.39): 56 data bytes
64 bytes from 10.7.17.39: icmp_seq=0 ttl=64 time=57.403 ms
64 bytes from 10.7.17.39: icmp_seq=1 ttl=64 time=13.697 ms
64 bytes from 10.7.17.39: icmp_seq=2 ttl=64 time=9.138 ms
64 bytes from 10.7.17.39: icmp_seq=3 ttl=64 time=119.632 ms

--- 10.7.17.39 ping statistics ---
4 packets transmitted, 4 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 9.138/49.968/119.632/44.416 ms
[vikrantyadav@Vikrants-MacBook-Pro CN-Project % ping -c 4 10.7.15.184
PING 10.7.15.184 (10.7.15.184): 56 data bytes
64 bytes from 10.7.15.184: icmp_seq=0 ttl=64 time=19.273 ms
64 bytes from 10.7.15.184: icmp_seq=1 ttl=64 time=6.381 ms
64 bytes from 10.7.15.184: icmp_seq=2 ttl=64 time=49.047 ms
64 bytes from 10.7.15.184: icmp_seq=3 ttl=64 time=71.168 ms

--- 10.7.15.184 ping statistics ---
4 packets transmitted, 4 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 6.381/36.467/71.168/25.314 ms
vikrantyadav@Vikrants-MacBook-Pro CN-Project %

```

Figure 3: Task A: Mac 1 to Mac 2 and Mac 4, 4 packets each, 0% loss.

```

mayankpillai@Mayanks-MacBook-Pro-2 CN Project % dig app.team1.test

; <<>> DiG 9.10.6 <<>> app.team1.test
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 37158
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;app.team1.test.                IN      A

;; ANSWER SECTION:
app.team1.test.                0      IN      A      10.7.17.39

;; Query time: 57 msec
;; SERVER: 10.7.13.235#53(10.7.13.235)
;; WHEN: Thu Oct 01 13:49:53 IST 2026
;; MSG SIZE rcvd: 59

```

Figure 4: Task B: dig app.team1.test from a client (Mac 3). The answer is the edge IP 10.7.17.39 and the SERVER line is our DNS server 10.7.13.235.

2 How the Configuration Works

2.1 DNS server (Mac 1, dnsmasq)

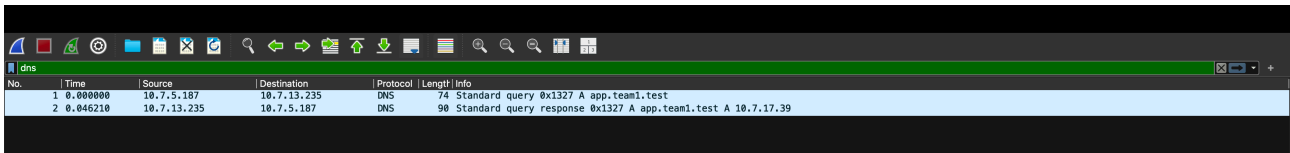
dnsmasq listens on Mac 1's LAN address (not only 127.0.0.1), so other machines can use it, and answers for our two names with the edge's IP.

```
# Mac 1 (DNS server) - Task B. Path: /opt/homebrew/etc/dnsmasq.conf
listen-address=10.7.13.235,127.0.0.1

# Both names point at the edge (Mac 2 / nginx)
address=/app.team1.test/10.7.17.39
address=/api.team1.test/10.7.17.39

interface=en0
```

In the capture below, the client (10.7.5.187, UDP port 58206) asks the DNS server (10.7.13.235, port 53) for `app.team1.test`; the answer is 10.7.17.39 with TTL 0.



The image shows a Wireshark packet capture window with the filter 'dns'. It displays two packets:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.7.5.187	10.7.13.235	DNS	74	Standard query 0x1327 A app.team1.test
2	0.046210	10.7.13.235	10.7.5.187	DNS	90	Standard query response 0x1327 A app.team1.test A 10.7.17.39

Figure 5: Wireshark filter dns: query (frame 1) and response (frame 2).

2.2 Backends (Mac 3 and Mac 4)

Each backend is a small Python HTTP server that listens on all interfaces (0.0.0.0), on port 3001 (A) or 3002 (B). It answers GET / and GET /api/status, and every response carries an `X-Backend: A` (or `B`) header and `Cache-Control: max-age=60`. The code is in `backend/backend_a.py` and `backend/backend_b.py`.

2.3 nginx edge: TLS termination and load balancing (Mac 2)

nginx is the single entry point. It terminates TLS on port 8443 with a certificate for `app.team1.test` and `api.team1.test` created with `mkcert` (approved by our faculty), and forwards requests round-robin to the two backends, so clients never learn the backend IPs.

```
worker_processes 1;

events {
    worker_connections 1024;
}

http {
    # Log which backend served each request (useful evidence for Task D / failure demos)
    log_format lb '$remote_addr [$time_local] "$request" $status '
        'upstream=$upstream_addr backend=$upstream_http_x_backend';
    access_log /opt/homebrew/var/log/nginx/access.log lb;

    # TASK D: Upstream Load Balancer (round-robin is the default policy)
    # max_fails/fail_timeout = passive health check: after 1 failure the backend
    # is skipped for 10s, so traffic keeps flowing to the healthy one.
    upstream backend_servers {
        server 10.7.5.187:3001 max_fails=1 fail_timeout=10s; # Mac 3 (Backend A)
        server 10.7.15.184:3002 max_fails=1 fail_timeout=10s; # Mac 4 (Backend B)
    }
}
```

```

# TASK E: HTTPS / TLS Server Block
server {
    # Listen on HTTPS port 8443
    listen 8443 ssl;
    http2 on;    # lets you show HTTP/2 (curl --http2 -I ...); remove if your nginx is < 1.25.1

    server_name app.team1.test api.team1.test;

    # TLS Certificate paths (mkcert files)
    ssl_certificate      /opt/homebrew/etc/nginx/app.team1.test.pem;
    ssl_certificate_key  /opt/homebrew/etc/nginx/app.team1.test-key.pem;

    # Modern SSL settings
    ssl_protocols TLSv1.2 TLSv1.3;

    # Failover: if one backend errors/times out, retry the request on the other
    proxy_connect_timeout 2s;
    proxy_read_timeout 10s;
    proxy_next_upstream error timeout http_502 http_503 http_504;

    # Both backends down -> clear 502 from the edge
    error_page 502 503 504 = @upstream_down;
    location @upstream_down {
        default_type text/plain;
        return 502 "502 Bad Gateway: edge is up, but no backend answered.\n";
    }

    location / {
        proxy_pass http://backend_servers;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

```

```

mayankpillai@Mayanks-MacBook-Pro-2 CN Project % curl -v https://app.team1.test:8443/api/status
* Host app.team1.test:8443 was resolved.
* IPv6: (none)
* IPv4: 10.7.17.39
* Trying 10.7.17.39:8443...
* Connected to app.team1.test (10.7.17.39) port 8443
* ALPN: curl offers h2,http/1.1
* (304) (OUT), TLS handshake, Client hello (1):
* CAfile: /etc/ssl/cert.pem
* CApath: none
* (304) (IN), TLS handshake, Server hello (2):
* (304) (IN), TLS handshake, Unknown (8):
* (304) (IN), TLS handshake, Certificate (11):
* (304) (IN), TLS handshake, CERT verify (15):
* (304) (IN), TLS handshake, Finished (20):
* (304) (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / AEAD-CHACHA20-POLY1305-SHA256 / [blank] / UNDEF
* ALPN: server accepted h2
* Server certificate:
* subject: 0=mkcert development certificate; OU=faizalil@Asads-MacBook-Pro-2.local (Faiz Ali)
* start date: Oct 1 07:10:34 2026 GMT
* expire date: Jan 1 07:10:34 2029 GMT
* subjectAltName: host "app.team1.test" matched cert's "app.team1.test"
* issuer: 0=mkcert development CA; OU=faizalil@Asads-MacBook-Pro-2.local (Faiz Ali); CN=mkcert faizalil@Asads-MacBook-Pro-2.local (Faiz Ali)
* SSL certificate verify ok.
* using HTTP/2
* [HTTP/2] [1] OPENED stream for https://app.team1.test:8443/api/status
* [HTTP/2] [1] [:method: GET]
* [HTTP/2] [1] [:scheme: https]
* [HTTP/2] [1] [:authority: app.team1.test:8443]
* [HTTP/2] [1] [:path: /api/status]
* [HTTP/2] [1] [user-agent: curl/8.7.1]
* [HTTP/2] [1] [accept: /*]
> GET /api/status HTTP/2
> Host: app.team1.test:8443
> User-Agent: curl/8.7.1
> Accept: /*
>
* Request completely sent off
< HTTP/2 200
< server: nginx/1.31.6
< date: Thu, 01 Oct 2026 08:39:35 GMT
< content-type: application/json
< x-backend: A
< cache-control: max-age=60
* Connection #0 to host app.team1.test left intact

```

Figure 6: Task E: `curl -v https://app.team1.test:8443/api/status` with no `-k`. TLS 1.3, certificate subject name matches `app.team1.test`, “SSL certificate verify ok”, HTTP 200 with `x-backend: A`.

```

mayankpillai@Mayanks-MacBook-Pro-2 CN Project % curl -i https://app.team1.test:8443/api/status
HTTP/2 200
server: nginx/1.31.6
date: Thu, 01 Oct 2026 10:04:38 GMT
content-type: application/json
x-backend: A
cache-control: max-age=60

{"backend": "A", "status": "ok"}
mayankpillai@Mayanks-MacBook-Pro-2 CN Project % curl -i https://app.team1.test:8443/api/status
HTTP/2 200
server: nginx/1.31.6
date: Thu, 01 Oct 2026 10:04:40 GMT
content-type: application/json
x-backend: A
cache-control: max-age=60

{"backend": "A", "status": "ok"}
mayankpillai@Mayanks-MacBook-Pro-2 CN Project % curl -i https://app.team1.test:8443/api/status
HTTP/2 200
server: nginx/1.31.6
date: Thu, 01 Oct 2026 10:04:41 GMT
content-type: application/json
content-length: 32
x-backend: B
cache-control: max-age=60

{"backend": "B", "status": "ok"}
mayankpillai@Mayanks-MacBook-Pro-2 CN Project % curl -i https://app.team1.test:8443/api/status
HTTP/2 200
server: nginx/1.31.6
date: Thu, 01 Oct 2026 10:04:42 GMT
content-type: application/json
x-backend: A
cache-control: max-age=60

{"backend": "A", "status": "ok"}
mayankpillai@Mayanks-MacBook-Pro-2 CN Project % curl -i https://app.team1.test:8443/api/status
HTTP/2 200
server: nginx/1.31.6
date: Thu, 01 Oct 2026 10:04:46 GMT
content-type: application/json
content-length: 32
x-backend: B
cache-control: max-age=60

{"backend": "B", "status": "ok"}
mayankpillai@Mayanks-MacBook-Pro-2 CN Project %

```

Figure 7: Task D: repeated requests to the same domain are served by both backends (`x-backend A` and `B`).

2.4 HTTP caching (Task F)

The backends send **Cache-Control: max-age=60**: a client may reuse the response for 60 seconds without asking the server again; after that it must fetch it again.

```
mayankpillai@Mayanks-MacBook-Pro-2 CN Project % curl -i https://app.team1.test:8443/api/status
HTTP/2 200
server: nginx/1.31.6
date: Thu, 01 Oct 2026 08:47:45 GMT
content-type: application/json
x-backend: A
cache-control: max-age=60

{"backend": "A", "status": "ok"}%
```

Figure 8: Cache-Control, Date and X-Backend headers in the response.

2.5 Packet evidence (Task G)

One `curl` request from Mac 3 (10.7.5.187) to the edge (10.7.17.39), captured in Wireshark.

TCP three-way handshake. The client's port 56982 connects to port 8443: SYN, SYN-ACK, ACK (frames 3 to 5), before any application data is sent.

No.	Time	Source	Destination	Protocol	Length	Info
3	0.848883	10.7.5.187	10.7.17.39	TCP	78	56982 → 8443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=244392986 TSecr=0 SACK_PERM
4	0.856165	10.7.17.39	10.7.5.187	TCP	78	8443 → 56982 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=64 TSval=4107782356 TSecr=244392986 SACK_PERM
5	0.856325	10.7.5.187	10.7.17.39	TCP	66	56982 → 8443 [ACK] Seq=1 Ack=1 Win=131776 Len=0 TSval=244392915 TSecr=4107782356

Figure 9: Filter `tcp.port==8443`: SYN, SYN-ACK, ACK.

TLS handshake. The ClientHello (frame 6, SNI `app.team1.test`) is answered by the ServerHello (frame 9), which selects TLS 1.3 with `TLS_CHACHA20_POLY1305_SHA256`. In TLS 1.3 the server certificate is encrypted, so it does not appear as a separate packet here; `curl -v` (above) shows it.

6	0.856648	10.7.5.187	10.7.17.39	TLSv1..	390	Client Hello (SNI=app.team1.test)
7	0.877386	10.7.17.39	10.7.5.187	TCP	66	[TCP Window Update] 8443 → 56982 [ACK] Seq=1 Ack=1 Win=131776 Len=0 TSval=4107782364 TSecr=244392915
8	0.877387	10.7.17.39	10.7.5.187	TCP	66	8443 → 56982 [ACK] Seq=1 Ack=325 Win=131456 Len=0 TSval=4107782364 TSecr=244392915
9	0.877389	10.7.17.39	10.7.5.187	TLSv1..	1514	Server Hello, Change Cipher Spec, Application Data, Application Data

Figure 10: ClientHello, then ServerHello with Change Cipher Spec.

Encrypted payload. After the handshake every packet is TLS “Application Data”. The HTTP headers and body are inside encrypted records, so Wireshark cannot show them; we use `curl -v` to show the headers.

10	0.877391	10.7.17.39	10.7.5.187	TLSv1..	329	Application Data, Application Data
11	0.877552	10.7.5.187	10.7.17.39	TCP	66	56982 → 8443 [ACK] Seq=325 Ack=1712 Win=130112 Len=0 TSval=244392936 TSecr=4107782375
12	0.109962	10.7.5.187	10.7.17.39	TLSv1..	72	Change Cipher Spec
13	0.110067	10.7.5.187	10.7.17.39	TLSv1..	124	Application Data
14	0.110256	10.7.5.187	10.7.17.39	TLSv1..	152	Application Data
15	0.110366	10.7.5.187	10.7.17.39	TLSv1..	140	Application Data
16	0.120982	10.7.17.39	10.7.5.187	TCP	66	8443 → 56982 [ACK] Seq=1712 Ack=549 Win=131264 Len=0 TSval=4107782419 TSecr=244392969
17	0.120985	10.7.17.39	10.7.5.187	TLSv1..	337	Application Data
18	0.120989	10.7.17.39	10.7.5.187	TLSv1..	337	Application Data
19	0.121061	10.7.5.187	10.7.17.39	TCP	66	56982 → 8443 [ACK] Seq=549 Ack=2254 Win=130560 Len=0 TSval=244392979 TSecr=4107782420
20	0.121127	10.7.17.39	10.7.5.187	TLSv1..	137	Application Data
21	0.121587	10.7.5.187	10.7.17.39	TCP	66	56982 → 8443 [ACK] Seq=549 Ack=2325 Win=131008 Len=0 TSval=244392980 TSecr=4107782422
22	0.121748	10.7.5.187	10.7.17.39	TLSv1..	97	Application Data
23	0.130408	10.7.17.39	10.7.5.187	TCP	66	8443 → 56982 [ACK] Seq=2325 Ack=580 Win=131264 Len=0 TSval=4107782430 TSecr=244392980
24	0.140501	10.7.17.39	10.7.5.187	TLSv1..	218	Application Data
25	0.140626	10.7.5.187	10.7.17.39	TCP	66	56982 → 8443 [ACK] Seq=580 Ack=2477 Win=130944 Len=0 TSval=244392999 TSecr=4107782440
26	0.141175	10.7.5.187	10.7.17.39	TLSv1..	90	Application Data

Figure 11: Only encrypted Application Data after the handshake.

3 Failure Demonstration: Stop One Backend

What we broke: we stopped Backend A (the Python process on Mac 3, 10.7.5.187:3001).

Before: requests to `https://app.team1.test:8443/api/status` were served by both backends (x-backend alternating between A and B; see the load-balancing figure above).

After: every response came from Backend B and still returned HTTP 200.

```
mayankpillai@Mayanks-MacBook-Pro-2 CN Project % curl -i https://app.team1.test:8443/api/status
HTTP/2 200
server: nginx/1.31.6
date: Thu, 01 Oct 2026 09:57:41 GMT
content-type: application/json
content-length: 32
x-backend: B
cache-control: max-age=60

{"backend": "B", "status": "ok"}%
mayankpillai@Mayanks-MacBook-Pro-2 CN Project % curl -i https://app.team1.test:8443/api/status
HTTP/2 200
server: nginx/1.31.6
date: Thu, 01 Oct 2026 09:57:44 GMT
content-type: application/json
content-length: 32
x-backend: B
cache-control: max-age=60

{"backend": "B", "status": "ok"}%
mayankpillai@Mayanks-MacBook-Pro-2 CN Project % curl -i https://app.team1.test:8443/api/status
HTTP/2 200
server: nginx/1.31.6
date: Thu, 01 Oct 2026 09:57:46 GMT
content-type: application/json
content-length: 32
x-backend: B
cache-control: max-age=60

{"backend": "B", "status": "ok"}%
mayankpillai@Mayanks-MacBook-Pro-2 CN Project %
```

Figure 12: Backend A stopped: three consecutive requests, all x-backend: B, HTTP/2 200.

Layer affected: the application / backend service layer behind the load balancer. Port 3001 on Mac 3 stopped accepting connections, so nginx stopped sending requests there and used Backend B. DNS, IP, the TCP connection to the edge and TLS were not affected: the client still resolved the name, connected to nginx and completed the TLS handshake. This is why a load balancer gives resilience.

Restored: we restarted Backend A on Mac 3 (`python3 backend/backend_a.py`). Afterwards the same requests were again served by both backends (the load-balancing screenshot was taken after the failure demonstrations).